# Monte Carlo Study of the Bias-Variance Tradeoff

## Introduction

This vignette demonstrates the core functions used for our Monte Carlo study of the bias-variance tradeoff in regression models. Currently, these functions are sourced directly from our `function.R` script.

```
library(AKMProject)
# Load our core functions
```

## 1. Generating Data

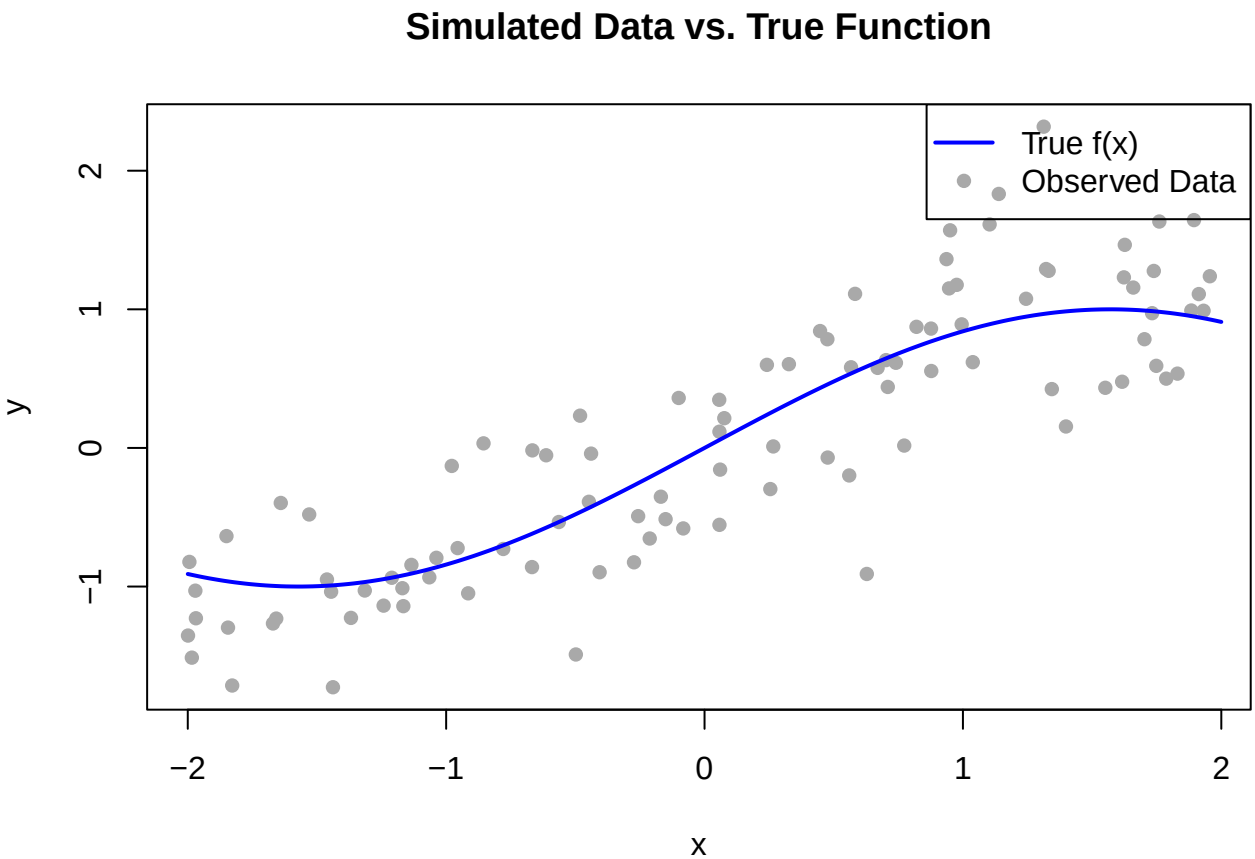
We start by generating a synthetic dataset. Our data-generating process relies on a true underlying function $f(x)$ and adds Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$.

```
set.seed(42)
# Define the true function (smooth version: sin(x))
f_true <- true_function("smooth")

# Generate 100 observations with noise level sigma = 0.5
train_data <- generate_data(n = 100, sigma = 0.5, f = f_true)

# Plot the generated data against the true function
x_seq <- seq(-2, 2, length.out = 200)
y_true <- f_true(x_seq)

plot(train_data$x, train_data$y, pch = 16, col = "darkgray",
     xlab = "x", ylab = "y", main = "Simulated Data vs. True Function")
lines(x_seq, y_true, col = "blue", lwd = 2)
legend("topright", legend = c("True f(x)", "Observed Data"),
       col = c("blue", "darkgray"), lwd = c(2, NA), pch = c(NA, 16))
```



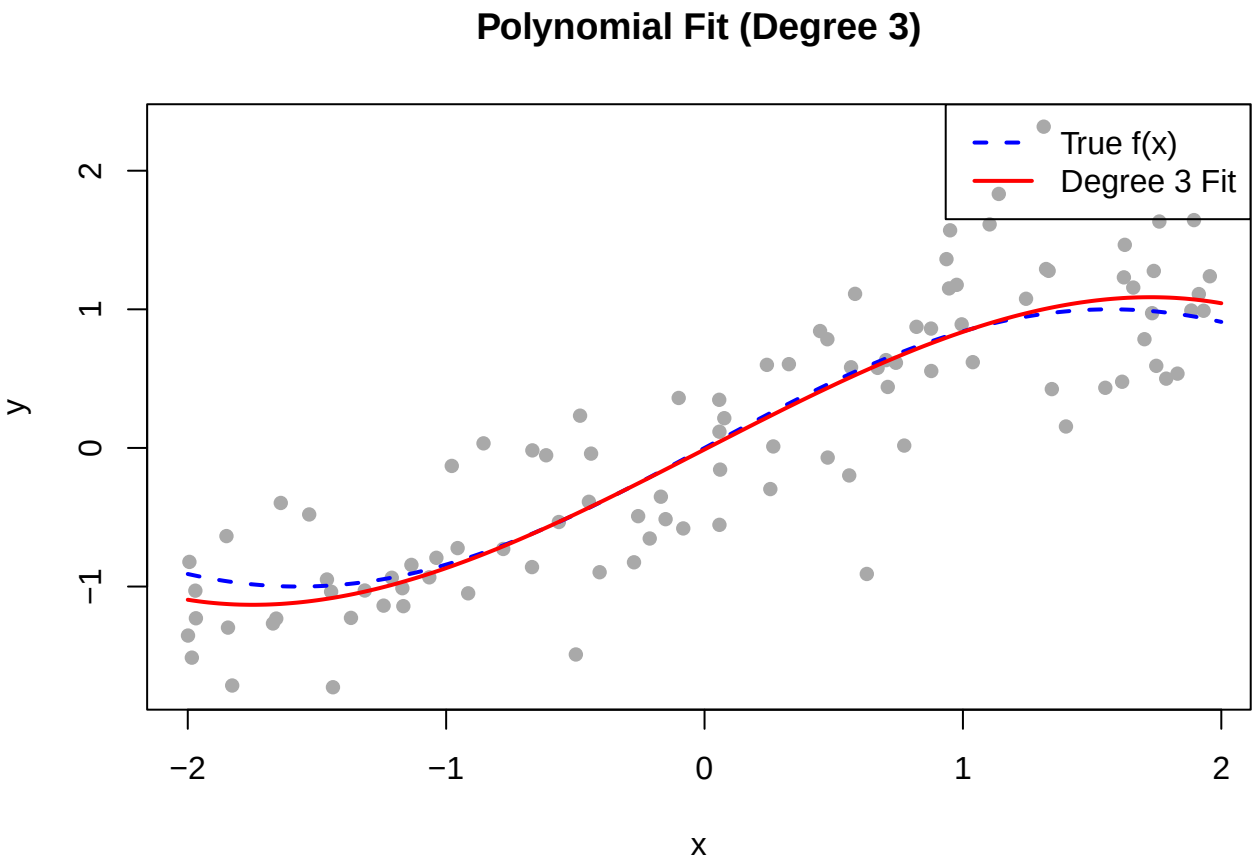

## 2. Fitting Polynomial Regression

Next, we demonstrate how to fit a polynomial regression model to this training data. We will fit a degree-3 polynomial.

```
# Fit a degree 3 polynomial
model_deg3 <- fit_polynomial(data = train_data, degree = 3)

# Predict over the grid to visualize the fit
y_pred_deg3 <- predict(model_deg3, newdata = data.frame(x = x_seq))

plot(train_data$x, train_data$y, pch = 16, col = "darkgray",
     xlab = "x", ylab = "y", main = "Polynomial Fit (Degree 3)")
lines(x_seq, y_true, col = "blue", lwd = 2, lty = 2)
lines(x_seq, y_pred_deg3, col = "red", lwd = 2)
legend("topright", legend = c("True f(x)", "Degree 3 Fit"),
       col = c("blue", "red"), lty = c(2, 1), lwd = 2)
```




### 3. Evaluating Performance

We can compute the Mean Squared Error (MSE) of our fitted model on the training set.

```
train_mse <- compute_mse(model = model_deg3, data = train_data)
cat("Training MSE for Degree 3:", round(train_mse, 4), "\n")
#> Training MSE for Degree 3: 0.2089
```

### 4. Monte Carlo Simulation

Finally, we run a small-scale Monte Carlo simulation to estimate the bias squared and variance for a given polynomial degree. *Note: In practice, $B$ should be much larger (e.g., 500 or 1000), but we use $B = 50$ here for demonstration speed.*

```
set.seed(42)
mc_results <- monte_carlo_bias_variance(B = 50, n = 100, sigma = 0.5,
                                        degree = 3, f = f_true)

cat("Estimated Bias Squared:", round(mc_results$bias2, 4), "\n")
#> Estimated Bias Squared: 3e-04
cat("Estimated Variance:", round(mc_results$variance, 4), "\n")
#> Estimated Variance: 0.0107
```

### Generative AI Usage Statement

We used generative AI tools to assist in writing the LaTeX code for the proposal. All statistical methods, project design decisions, and final content were reviewed and validated by team members. We did not use

any generative AI tools to generate results, simulations, or analyses.

## References


Chang, Winston, Joe Cheng, JJ Allaire, Carson Sievert, Barret Schloerke, Yihui Xie, Jeff Allen, Jonathan McPherson, Alan Dipert, and Barbara Borges. 2024. *Shiny: Web Application Framework for r*. https://CRAN.R-project.org/package=shiny.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media. https://hastie.su.domains/ElemStatLearn/.

R Core Team. 2024. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Voss, Jochen. 2013. *An Introduction to Statistical Computing: A Simulation-Based Approach*. John Wiley & Sons. https://onlinelibrary.wiley.com/doi/book/10.1002/9781118593124.